

Computability: Recitation 10

June 8nd, 2026

1 KNAPSACK

Definition 1 (KNAPSACK). *Given a set of items*

$$I = \{1, 2, \dots, n\},$$

where each item i has a weight $w_i \in \mathbb{Z}^+$ and a value $v_i \in \mathbb{Z}^+$, together with a capacity bound $W \in \mathbb{Z}^+$ and a target value $V \in \mathbb{Z}^+$, the KNAPSACK decision problem asks whether there exists a subset $I' \subseteq I$ such that

$$\sum_{i \in I'} w_i \leq W$$

and

$$\sum_{i \in I'} v_i \geq V.$$

Formally,

$$\text{KNAPSACK} = \left\{ \langle (w_1, v_1), \dots, (w_n, v_n), W, V \rangle : \exists I' \subseteq I \text{ such that } \sum_{i \in I'} w_i \leq W \text{ and } \sum_{i \in I'} v_i \geq V \right\}.$$

Proposition 2. *KNAPSACK is NP-complete.*

Proof. We prove NP-completeness by showing: (1) KNAPSACK $\in$ NP, and (2)

KNAPSACK is NP-hard by reducing SUBSET-SUM to KNAPSACK.

1.1 Part 1: KNAPSACK $\in$ NP

We construct a polynomial-time verifier V for SUBSET-SUM.

Input: (w, c) where $w = \langle (w_1, v_1), \dots, (w_n, v_n), W, V \rangle$ and the certificate c is a subset of items I' .

Verification Algorithm:

1. We verify that the selected items indeed form a valid subset of the given set of items (W, V) .
2. Compute

$$\sum_{i \in I'} w_i.$$

3. Compute

$$\sum_{i \in I'} v_i.$$

4. Accept iff

$$\sum_{i \in I'} w_i \leq W$$

and

$$\sum_{i \in I'} v_i \geq V.$$

Both sums can be computed in polynomial time, therefore KNAPSACK $\in$ NP.

1.2 Part 2: NP-Hardness via Reduction from SUBSET-SUM

We show

$$\text{SUBSET-SUM} \leq_p \text{KNAPSACK}.$$

Let

$$\langle S, t \rangle$$

be an instance of SUBSET-SUM, where

$$S = \{s_1, s_2, \dots, s_n\}.$$

Construct the following KNAPSACK instance:

- For each element s_i , create an item whose weight and value are both equal to s_i :

$$w_i = v_i = s_i.$$

- Set the knapsack capacity to

$$W = t.$$

- Set the required value to

$$V = t.$$

1.2.1 Correctness

Forward Direction ($\Rightarrow$):

Suppose there exists a subset

$$S' \subseteq S$$

such that

$$\sum_{s \in S'} s = t.$$

Choose the corresponding items in the KNAPSACK instance.

Since each item's weight equals its value,

$$\sum w_i = t$$

and

$$\sum v_i = t.$$

Therefore the capacity and value constraints are satisfied.

Backward Direction ($\Leftarrow$):

Suppose the KNAPSACK instance has a feasible solution.

Then

$$\sum w_i \leq t$$

and

$$\sum v_i \geq t.$$

Since every item satisfies

$$w_i = v_i,$$

the two sums are equal:

$$\sum w_i = \sum v_i.$$

Hence

$$t \leq \sum w_i = \sum v_i \leq t.$$

Therefore

$$\sum w_i = t.$$

The corresponding elements of S form a subset whose sum is exactly t , yielding a solution to the SUBSET-SUM instance.

1.2.2 Polynomial-Time Analysis

The reduction creates one item for each element of S and performs only a linear amount of work.

Therefore the reduction runs in polynomial time.

1.3 Conclusion

Since $\text{KNAPSACK} \in \text{NP}$ and $\text{SUBSET-SUM} \leq_p \text{KNAPSACK}$, KNAPSACK is NP-hard.

Therefore, **KNAPSACK is NP-complete.**

□

2 Question 2024-A

Let Σ be an alphabet, let $\#$ be a symbol not in Σ , and let $L \subseteq \Sigma^*$ be a language. Define the language $\text{AtLeastTwo}(L)$ as follows:

$$\text{AtLeastTwo}(L) = \left\{ w_1\#w_2\#\cdots\#w_n \mid \begin{array}{l} n \geq 2, w_i \in \Sigma^* \text{ for all } 1 \leq i \leq n, \\ \text{and there exist two distinct indices } 1 \leq i < j \leq n \\ \text{such that } w_i \in L \text{ and } w_j \in L \end{array} \right\}.$$

Claim: If L is NP-Complete, then $\text{AtLeastTwo}(L)$ is also NP-Complete.

Solution:

The claim is correct; the class NPC is closed under the AtLeastTwo operator.

First direction. (NP is closed under AtLeastTwo) Let Σ be an alphabet that does not include $\#$, and consider a language $L \subseteq \Sigma^*$ that is in NP. It is not hard to verify that the following identity holds:

$$\text{AtLeastTwo}(L) = (\Sigma^* \cdot \#)^* \cdot L \cdot \{\#\} \cdot (\Sigma^* \cdot \#)^* \cdot L \cdot (\# \cdot \Sigma^*)^*.$$

Observe that $(\Sigma \cdot \#)^*$, $(\# \cdot \Sigma^*)^*$, and $\{\#\}$ are all regular languages and hence belong to $DTIME(n) \subseteq P \subseteq NP$, and by assumption $L \in NP$. Thus, $AtLeastTwo(L)$ is a finite concatenation of NP languages, and by the closure of NP under finite concatenations, we conclude that

$$AtLeastTwo(L) \in NP.$$

Second direction. (If L is NP-hard, then so is $AtLeastTwo(L)$) If L is NP-hard, then the mapping

$$w \mapsto w\#w$$

is a polynomial-time reduction from L to $AtLeastTwo(L)$. Therefore,

$$L \leq_p AtLeastTwo(L).$$

Since the reduction is clearly computable in polynomial time, it follows that if L is NP-hard, then so is $AtLeastTwo(L)$.

3 NP vs coNP

Definition 3. $coNP = \{L : \bar{L} \in NP\}$

Because of the asymmetric nature of NTMs, if a language L is NP, it is not clear whether $\bar{L} \in NP$. From the perspective of verifiers, we know that if $L \in NP$, then we can “convince” that $x \in L$ by providing a witness, but we don’t necessarily know how to convince that $x \notin L$ (think of SAT for example. We don’t know of a short way to convince that a Boolean formula is not satisfiable). This is the reason we define the class coNP of languages whose complement is in NP. It is generally believed that $NP \neq coNP$, but proving that would prove that $P \neq NP$. Indeed, as P is closed under complementation, $NP = P$ implies that NP is closed under complementation, and so $coNP = NP$. Thus, conversely we have that $NP \neq coNP$ implies that $P \neq NP$.

Similar to NP, we define hardness and completeness with respect to the class coNP.

Definition 4. A language L is coNP-hard if for every $K \in coNP$ it holds that $K \leq_p L$. It is coNP-complete if also $L \in coNP$.

Claim 5. L is NP-hard iff $\bar{L}$ is coNP-hard.

Corollary 6. L is NP-complete iff $\bar{L}$ is coNP-complete.

Proof. On one direction, if L is NP-hard, then for every $K \in coNP$, it holds that $\bar{K} \leq_p L$. The same reduction shows that also $K \leq_p \bar{L}$, thus $\bar{L}$ is coNP-hard. The other direction is similar. $\square$

3.1 Boolean formulas

A Boolean formula φ is called a tautology if every assignment of true/false values to variables yields a true value. It is called a contradiction if every assignment yields a false value. Consider the following languages.

$$\begin{aligned} \text{CNF} - \text{CONTRADICTION} &= \{\langle \varphi \rangle : \varphi \text{ is a CNF formula that is a contradiction}\} \\ \text{CNF} - \text{TAUTOLOGY} &= \{\langle \varphi \rangle : \varphi \text{ is a CNF formula that is a tautology}\} \end{aligned}$$

Interestingly, the fact that we consider formulas in conjunctive normal form places the two languages in different complexity classes. First, it is not hard to see that CNF – TAUTOLOGY is solvable in polynomial time. The general idea is that a CNF formula is a tautology iff each of its clauses is a tautology (Why?), and checking if a single clause is a tautology is easy.

However, as $\overline{\text{CNF} - \text{SAT}} = \text{CNF} - \text{CONTRADICTION}$, the language CNF – CONTRADICTION turns out to be coNP-complete.

Remark: Note that $\text{CNF} - \text{TAUTOLOGY} \in P$ is due to the fact that we are considering CNF formulas. That is, the language $\text{TAUTOLOGY} = \{\langle \varphi \rangle : \varphi \text{ is a tautology}\}$, that considers general formulas is coNP-complete. Indeed, the mapping $\langle \varphi \rangle \mapsto \langle \neg \varphi \rangle$ is a reduction from CNF – CONTRADICTION to TAUTOLOGY that is computable in polynomial time.

3.2 $NP \cap coNP$

Now that we defined the class $coNP$, it makes sense investigating the relation between NP and $coNP$. For instance, are there languages $L \in NP \cap coNP$? – Clearly there are, in fact, $P \subseteq NP \cap coNP$. But that is a bit too easy, what about “harder” languages that are in $NP \cap coNP$. The following theorem demonstrates a consequence of such a scenario.

Theorem 7. *If there is a language $L \in NP \cap coNP$ such that L is NP -hard then $NP = coNP$.*

Proof. Let $L' \in NP$, then $L' \leq_p L$, and since $L \in coNP$ it follows that $L' \in coNP$. Thus, $NP \subseteq coNP$ from which it follows that $NP = coNP$. $\square$

In Figure 1, the 3 possibilities for the relationship between P , NP and $coNP$ are depicted.

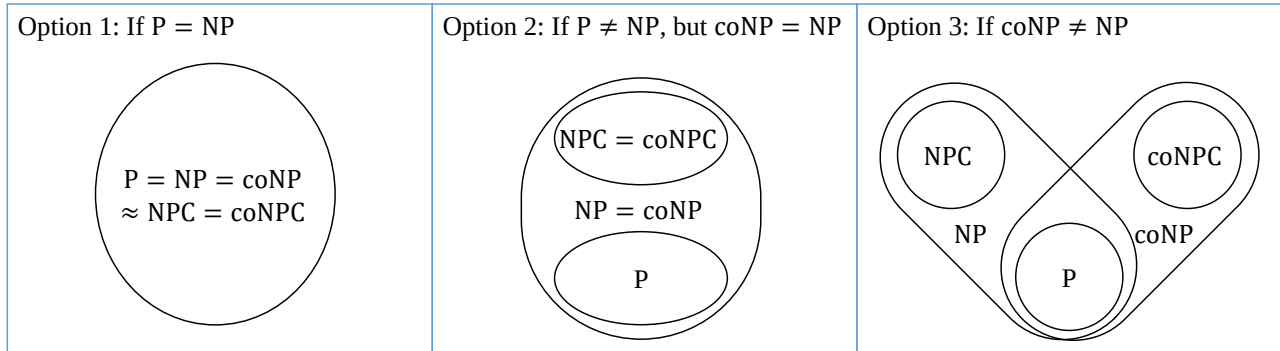


Figure 1: The 3 possibilities for the relationship between $PTIME$, NP and $coNP$. Make sure you understand all the relationships. The reason for the $\approx$ notation in Option 1 is that NP -complete does not contain $\emptyset$ and Σ^* . Note that it is not known whether $P = NP \cap coNP$ or not.